

Support Intelligence Agent

Implementation Documentation Set

Consolidated implementation documentation for use as the project source of truth in Cursor.

DOCUMENT 1 — PRODUCT REQUIREMENTS DOCUMENT

Product Name

Support Intelligence Agent

Product Definition

A role-based AI intelligence platform supporting both customers seeking information about their own account, orders, tickets, agreements, and applicable policies, and authorized Internal Support users investigating customer issues, operational data, policies, known issues, and performing controlled operational actions.

Platform Capabilities

- structured operational data
- unstructured policies and agreements
- reliable source resolution
- multi-step AI reasoning
- role-based authorization
- human confirmation for state-changing actions

Primary Objectives

- Ask natural-language questions.
- Retrieve answers from relevant policies and documents.
- Query structured operational data.
- Combine multiple sources to answer complex questions.
- Respect customer-specific agreements.
- Resolve source conflicts safely.
- Avoid using deprecated or unreliable sources for current decisions.
- Identify missing information rather than guessing.
- Escalate cases requiring human judgment.
- Perform approved state-changing actions only after explicit confirmation.
- Detect recurring operational issues and risks.

User Types — Customer

A customer is associated with a specific account. Dataset examples: Northstar Logistics — ACCT-001; LumenWorks — ACCT-002. Customer access is restricted to their own account, orders, tickets, applicable agreement, and applicable general policies/product documentation. A customer must never access another customer's data.

User Types — Internal Support

Authorized internal users can investigate orders and tickets, retrieve applicable agreements, review policies, identify source conflicts, investigate recurring issues, create approved mock operational actions, and access the Issue Intelligence dashboard. MVP role: INTERNAL_SUPPORT. The permissions system should remain extensible.

Core Functional Requirements — Conversational AI

- accept natural-language queries
- determine intent
- identify relevant entities
- use one or more tools
- combine results
- return a grounded answer
- provide source information where relevant

Core Functional Requirements — Role-Based Access

Authorization is enforced in the backend and tool layer. The LLM does not decide whether a user may access data. Restricted data must never reach the model.

Core Functional Requirements — Source-Based Answers

Source types include active customer agreements, current policies/SOPs, current product documentation, historical tickets/internal context, and deprecated documents. The Source Resolution Engine determines applicability.

Core Functional Requirements — Multi-Step Reasoning

Order/account investigation may require: retrieve order → identify customer → retrieve applicable agreement → retrieve applicable policy → resolve source precedence → calculate outcome → return grounded answer.

Core Functional Requirements — State-Changing Actions

Request → prepare proposed action → create PendingAction → show confirmation → user explicitly confirms → backend executes action → create audit log.

Core Functional Requirements — Proactive Issue Intelligence

- recurring issues
- repeated ticket themes
- known-issue correlations
- SLA risks
- tickets approaching SLA deadlines
- cross-customer patterns
- unusual spikes
- source conflicts requiring verification

Non-Goals

- real carrier API integrations
- real production ticketing integrations
- full enterprise SSO
- automated actions without confirmation
- autonomous business decisions
- fine-tuning
- multi-agent architecture
- Kubernetes
- microservices
- real-time large-scale streaming analytics

DOCUMENT 2 — TECHNICAL REQUIREMENTS DOCUMENT

Architecture Principle

Hybrid architecture: Django Backend + OpenAI Responses API for reasoning/tool calling + Custom Django Orchestration for authorization, source resolution, confirmation control and state management + LangChain for document ingestion, chunking, embeddings and retrieval. Business-critical safety decisions must not depend on autonomous framework behavior.

Technology Stack

Frontend: React, Vite, TypeScript. Backend: Python, Django, Django REST Framework. AI: OpenAI Responses API, OpenAI tool/function calling, LangChain for the RAG pipeline. Database: PostgreSQL + pgvector. Object Storage: AWS S3. Deployment: AWS + Docker. Compute services will be chosen according to available Free Tier credits.

Major Backend Components

authentication/, users/, accounts/, orders/, tickets/, documents/, agent/, source_resolution/, actions/, issue_intelligence/, observability/, audit/

Agent Architecture

User Query → Authentication Context → Intent/Entity Understanding → Tool Planning → Tool Execution → Source Resolution → Missing Information/Conflict Check → Decision: Answer / Ask for information / Request verification / Prepare action.

Agent Safety Principle

The model may choose useful tools, identify information needs, and summarize evidence. The backend decides authorization, data access, source applicability, action execution, and confirmation validity.

DOCUMENT 3 — APPLICATION FLOW DOCUMENT

Login Flow — Customer

Login → backend authenticates → determine account_id → create authenticated session → all tools automatically scoped to account. Example: Northstar Customer → ACCT-001.

Login Flow — Internal Support

Internal authorization/login → backend validates authorization → assign INTERNAL_SUPPORT role → grant authorized operational scope.

Normal Query Flow

User submits query → create user context → identify intent → extract entities → determine required tools → execute authorized tools → resolve sources → check completeness/conflicts → generate final response → store conversation and observability data.

Example: Cancellation Query

Can ORD-1001 be cancelled without a fee? → identify order ID → get order data → identify account → domain=CANCELLATION → retrieve customer agreement → retrieve current cancellation SOP → run Source Resolution Engine → determine applicable rule → return answer with reasoning summary and sources.

Customer Access Violation Flow

Request → tool receives user context → authorization check → requested account outside allowed scope → reject tool request → no restricted data returned → explain access limitation.

State-Changing Action Flow

User requests action → agent determines action is required → validate authorization → validate required information → create PendingAction → return proposed action → user explicitly confirms → execute backend action → PendingAction=EXECUTED → create AuditLog → return result.

Conflict Flow

Multiple sources → check applicability → status → temporal validity → authority → specificity → explicit override? If yes, apply override. If no, retrieve additional information if available; otherwise detect genuine conflict and require verification, or resolve if evidence is sufficient.

DOCUMENT 4 — UI/UX SPECIFICATION

Main Application Structure

Header + Sidebar + Main Content. Sidebar contains conversations/navigation; main content contains Chat or Intelligence views.

Main Sections

Customer: CHAT. Internal Support: CHAT and ISSUE INTELLIGENCE.

Chat Interface

- user messages
- AI responses
- source references
- tool activity
- missing-information requests
- conflict/verification status
- action confirmation cards

Tool Activity

Never expose hidden chain-of-thought. Show concise status such as: Searching applicable policies; Looking up order; Checking customer agreement; Resolving applicable terms.

Source Display

Show sources used, for example: Northstar Enterprise Agreement; Cancellation & Service Credit SOP; ORD-1001 operational record.

Decision Status

- RESOLVED
- OVERRIDE APPLIED
- MORE INFORMATION REQUIRED
- VERIFICATION REQUIRED
- HUMAN JUDGMENT REQUIRED

Action Confirmation Card

Show proposed action, reason, and Cancel/Confirm controls. Confirm calls a backend confirmation endpoint.

Issue Intelligence Dashboard

- SLA Risks
- Recurring Issues
- Cross-Customer Patterns
- Known Issue Correlations
- Source Conflicts
- Action Failures

DOCUMENT 5 — BACKEND AND DATABASE SCHEMA

Users

id, email, name, role, account_id nullable, is_active, created_at, updated_at. Roles: CUSTOMER, INTERNAL_SUPPORT.

Accounts

id, name, account_code, status, created_at.

Orders

Imported from supplied operational dataset. Index for order_id, account_id, status, carrier, created_at.

Tickets

Imported from supplied dataset. Index for ticket_id, account_id, severity, status, category, created_at.

SourceDocument

id, name, source_type, status, authority_level, scope_type, account_id nullable, effective_date, expiry_date, supersedes_document_id nullable, created_at, updated_at.

DocumentChunk

id, document_id, content, embedding, section_title, page_reference, chunk_status, chunk_effective_date, chunk_expiry_date, metadata, created_at. Chunk-level metadata is required because a current document may contain active, resolved, monitoring, and historical items.

ChatSession

id, user_id, title, created_at, updated_at.

ChatMessage

id, session_id, role, content, metadata, created_at.

PendingAction

id, session_id, user_id, action_type, payload, reason, status, created_at, expires_at, confirmed_at, executed_at. Statuses: DRAFT, AWAITING_CONFIRMATION, CONFIRMED, EXECUTED, CANCELLED, EXPIRED, FAILED.

ActionAuditLog

id, pending_action_id, user_id, action_type, request_payload, result, status, created_at.

Escalations

id, ticket_id, account_id, severity, reason, status, created_by, created_at.

FollowUpTasks

id, account_id, ticket_id, title, description, status, created_by, created_at.

Observability Events

id, session_id, user_id, event_type, tool_name, duration_ms, status, metadata, created_at. Track AGENT_REQUEST, TOOL_CALL, TOOL_SUCCESS, TOOL_FAILURE, SOURCE_CONFLICT, AUTHORIZATION_DENIED, PENDING_ACTION_CREATED, ACTION_CONFIRMED, ACTION_EXECUTED, ACTION_FAILED.

DOCUMENT 6 — AI INTELLIGENCE, SOURCE RESOLUTION AND RELIABILITY

Core Principle

The agent must not treat all retrieved sources as equally trustworthy. Resolution sequence: Applicability → Status → Temporal Validity → Authority → Specificity → Explicit Override → Conflict Detection.

Source Hierarchy — Level 1

Active Customer Agreement. Authority 100. Customer-specific. ACTIVE. Highest authority when applicable, active, and relevant.

Source Hierarchy — Level 2

Current Policy/SOP. Authority 80. General. CURRENT. Used unless superseded by an applicable agreement.

Source Hierarchy — Level 3

Current Product Documentation. Authority 70. Product/domain-specific. CURRENT. Used for capabilities, limitations, known issues, and workarounds.

Source Hierarchy — Level 4

Historical Tickets/Internal Notes. Authority 20. CONTEXT_ONLY. May provide historical context and pattern evidence, but must not independently establish current policy or contractual terms.

Source Hierarchy — Level 5

Deprecated Documents. Authority 0. DEPRECATED. Must not be used for current decisions; may be retained for historical explanation.

Temporal Validity

A source must satisfy $\text{effective_date} \leq \text{reference_date}$ and, where applicable, $\text{reference_date} \leq \text{expiry_date}$. For dataset-based questions, use the dataset snapshot/reference time. Temporal validity is not the same as authority; an older active customer agreement may override a newer general policy.

Specificity

Customer-specific rules are more specific than global rules when both address the same domain, but only apply when valid and explicitly relevant.

Explicit Override Rule

- the higher-authority source applies
- the source is active/current
- the relevant domain matches
- the source explicitly replaces, overrides, or defines different terms for the general rule

Missing Information Policy

1) Retrieve through an existing tool if possible. 2) Retrieve another authoritative source if it can resolve the gap. 3) Ask the user only for information they can reasonably provide. 4) If internal verification is required, explain the limitation and recommend verification. Never ask for information already available through internal tools.

Genuine Conflict Definition

A genuine conflict exists when applicable authoritative sources provide incompatible conclusions and no explicit precedence rule resolves them. The system must not choose randomly.

Resolution Outcomes

- RESOLVED — sources agree or evidence produces one valid answer.

- **OVERRIDE_APPLIED** — a higher-authority applicable source explicitly overrides a lower-level source.
- **NEEDS_MORE_INFORMATION** — a required fact is missing; retrieve before asking the user.
- **CONFLICT_REQUIRES_VERIFICATION** — authoritative sources genuinely disagree; do not execute consequential action.
- **HUMAN_JUDGMENT_REQUIRED** — discretionary approval, goodwill exception, unsupported exception, or business judgment is required; prepare escalation/follow-up instead of inventing an answer.

Example: Northstar Cancellation

Query → get ORD-1001 → identify ACCT-001 → retrieve Northstar agreement → retrieve current cancellation SOP → check applicability/validity → agreement explicitly overrides default → **OVERRIDE_APPLIED**.

Example: Incomplete Service Credit

Retrieve order and delay information → carrier fault unknown → check available internal tools → if unavailable, **NEEDS_MORE_INFORMATION** → request carrier verification. Do not promise a credit while required facts remain unknown.

DOCUMENT 7 — AGENT TOOLS, ORCHESTRATION, OBSERVABILITY AND AWS

Tool 1 — Document Search

Retrieve policies, agreements, product documentation, and known issues. Conceptual inputs: query, domain, account_id, user_context. Backend applies metadata filters before retrieval.

Tool 2 — Structured Data Lookup

Retrieve accounts, orders, tickets, and related records. Examples: get_order(order_id), get_account(account_id), get_tickets(account_id), get_ticket(ticket_id), get_related_tickets(...). The LLM must not receive unrestricted database access.

Tool 3 — Calculation Tool

Perform deterministic calculations: service credit, time/SLA calculations, delay duration, and eligibility-related calculations. Business-critical calculations should be performed deterministically by the backend.

Tool 4 — State-Changing Action Tool

Examples: prepare_escalation(), prepare_follow_up(), execute_confirmed_action(). The model must not directly execute irreversible actions.

Tool 5 — Issue Intelligence Tool

Detect recurring issues, repeated patterns, known-issue correlations, SLA risks, and cross-customer problems. Primarily available to INTERNAL_SUPPORT.

Tool Authorization Rule

Every tool receives user_id, role, account_id, and allowed_account_ids. The tool authorizes before querying data. Never rely solely on a prompt instruction not to reveal unauthorized information.

OpenAI + Django Execution Model

User Query → Django creates execution context → OpenAI Responses API → model requests tool → Django validates request → tool executes → result returned to model → Source Resolution Engine evaluates evidence → model produces grounded response. LangChain handles document loading, chunking, embedding, and vector retrieval.

Observability

- total requests
- model response time
- tool usage
- tool failures
- retrieval count
- source conflicts
- authorization denials
- pending actions
- action confirmation rate
- action failures

AWS Architecture

React/Vite → AWS Amplify → Django Backend → PostgreSQL + pgvector, with S3 for source files.

Deployment Acceptance Criteria

- Customer login/scope
- Internal support access

- AI chat
- Document retrieval
- Structured data lookup
- Source resolution
- Explicit override handling
- Missing-information handling
- Conflict handling
- Pending action confirmation
- At least one state-changing mock action
- Issue Intelligence
- Observability logging

RECOMMENDED IMPLEMENTATION ORDER

1. Backend + Database Schema
2. Authentication + RBAC
3. Data ingestion
4. Document ingestion + pgvector
5. Source Resolution Engine
6. Individual tools
7. OpenAI agent orchestration
8. Pending actions + audit logs
9. Frontend chat
10. Issue Intelligence
11. Testing
12. Docker
13. AWS deployment